

MMI S5 – Dispositifs interactifs – TD1 – Flux média et capture

1) API Media Capture and Streams

L'API **Media Capture and Streams** fournit le support pour décrire et manipuler des flux de données média, typiquement audio et/ou vidéo, en provenance de périphériques d'entrée (typiquement caméras, micros) ou de sortie, ou d'un partage d'écran. Elle contient deux interfaces principales : **MediaDevices** (sources des données média) et **MediaStream** (flux de données média).

1.1) Lister les sources et les contraintes disponibles

La propriété `navigator.mediaDevices` instancie un objet **MediaDevices** permettant de lister et d'accéder à l'ensemble des sources de données média disponibles dans le navigateur web.

La méthode `enumerateDevices()` de **MediaDevices** permet de **lister les sources**, elle retourne une promesse, celle d'un tableau d'objets **MediaDeviceInfo**, chacun fournissant des informations sur une source (type, label, identifiant) :

```
// Vérifier le support et lister les sources
if (!navigator.mediaDevices?.enumerateDevices) {
  console.log("enumerateDevices() not supported.");
} else {
  navigator.mediaDevices.enumerateDevices()
    .then((devices) => {
      devices.forEach((device) => {
        console.log(`${device.kind}: ${device.label},`
          + ` id = ${device.deviceId}`);
      });
    })
    .catch((err) => { console.error(`${err.name}: ${err.message}`); });
}
```

Le label et l'identifiant d'une source ne sont pas nécessairement disponibles, cela dépend des permissions persistantes accordées à la source et de son activité.

Certaines propriétés de la source sont **contraignables**, par exemple le nombre d'images par secondes (*frame per seconds, frame rate*) d'une caméra vidéo est paramétrable. Le navigateur reconnaît certaines de ces propriétés (principalement en fonction du navigateur et de la source). La méthode `getSupportedConstraints()` de **MediaDevices** permet de lister ces propriétés :

```
// Lister les propriétés contraignables acceptées par le navigateur
const constraints = navigator.mediaDevices.getSupportedConstraints();
for (const constraint of Object.keys(constraints))
  console.log(constraint);
```

Exercice 1 : Testez les codes ci-dessus, avec Firefox et Chrome.

1.2) Produire un flux et le diffuser

La méthode `getUserMedia(constraints)` de `MediaDevices` permet de **produire un flux de données**, avec des contraintes imposées aux sources. Elle retourne une promesse, celle d'un objet `MediaStream` décrivant le flux. Les contraintes contiennent les sources à utiliser :

```
// Produire un flux vidéo
const constraints = { audio: false, video: true };
navigator.mediaDevices.getUserMedia(constraints)
  .then((stream) => { /* utiliser le flux */ })
  .catch((err) => { /* gérer les erreurs */ });
```

La **diffusion du flux** dépend du type des sources, typiquement un élément HTML `video` pour un flux vidéo : `<video id="video">Flux vidéo non-disponible.</video>`

```
// Produire un flux vidéo (sans audio) et le diffuser
const constraints = { audio: false, video: true };
let video = document.querySelector("video");
navigator.mediaDevices.getUserMedia(constraints)
  .then((stream) => { /* diffuser le flux */
    video.srcObject = stream;
    video.onloadedmetadata = (e) => { video.play(); };
  })
  .catch((err) => { /* gérer les erreurs */ });
```

Exercice 2 : Testez le code ci-dessus. Remarquez que les dimensions (attributs HTML `width` et `height`) de l'élément `<video>` prennent automatiquement celles fournies par la caméra.

Pour que la taille de la vidéo dans la page soit éventuellement précisée en HTML avec l'attribut `width` ou `height`, par exemple :

```
<video id="video" width="256">Flux vidéo non-disponible.</video>
```

La valeur de l'autre attribut (`height` dans l'exemple) est calculé à partir de la largeur et de la hauteur des données fournies par la caméra, accessibles avec les propriétés `videoHeight` et `videoWidth` de `HTMLVideoElement` :

```
// Produire un flux vidéo (sans audio) et le diffuser
const constraints = { audio: false, video: true };
let video = document.querySelector("video");
navigator.mediaDevices.getUserMedia(constraints)
  .then((stream) => { /* diffuser le flux */
    video.srcObject = stream;
    video.onloadedmetadata = (e) => {
      if (video.hasAttribute('width')) {
        const width = video.getAttribute('width');
        const height = width * video.videoHeight / video.videoWidth;
        video.setAttribute('height', height);
      }
      else if (video.hasAttribute('height')) { /* ... */ }
      video.play();
    }
  })
  .catch((err) => { /* gérer les erreurs */ });
```

```
    };  
  })  
  .catch((err) => { /* gérer les erreurs */ });
```

Exercice 3 : Complétez (le cas de la hauteur) et testez le code ci-dessus.

1.3) Pistes et propriétés des sources

Un flux `MediaStream` est composé de 0 ou plusieurs **pistes** (comme audio et vidéo). Chaque piste est une instance de **MediaStreamTrack**. L'ensemble des pistes est représenté par un tableau. Les méthodes suivantes de `MediaStream` permettent d'accéder aux pistes :

- `getTracks()` retourne toutes les pistes (un tableau)
- `getTrackById(id)` retourne la piste d'identifiant `id`
- `getVideoTracks()` retourne un tableau restreint aux pistes vidéo (un tableau)
- `getAudioTracks()` retourne un tableau restreint aux pistes audio (un tableau)

Par exemple, dans l'exercice 3 ci-dessus, `stream.getTracks()[0]` est la piste vidéo.

Chaque piste `MediaStreamTrack` possède des **propriétés** et des **propriétés contraignables** relatives à la source, ainsi que la possibilité de **stopper** le flux. Sans explorer d'autres pages, lisez la référence de `MediaStreamTrack` : <https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack>.

Exercice 4 : A partir de la référence ci-dessus, complétez le code de l'exercice 3 pour afficher dans la console web, pour chaque piste composant le flux, les informations suivantes :

- le type (`kind`), le label et l'identifiant (similaire à ce que nous avons affiché dans la console à l'exercice 1).
- Les propriétés contraignables avec leurs valeurs acceptées (`getCapabilities()`) et leurs valeurs courantes (`getSettings()`) :

<https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack/getCapabilities>

<https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack/getSettings>

Exercice 5 : Complétez le code de l'exercice 4 avec les fonctionnalités suivantes :

- Le flux vidéo est produit et diffusé lorsque l'utilisateur (U) clique sur un bouton.
- L'U peut stopper la production/diffusion du flux avec un bouton.
- L'U peut mettre en pause la production/diffusion du flux avec un bouton, voir <https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement>
- L'U peut passer en mode plein écran avec un bouton, voir référence https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API

Les boutons sont placés en-dessous de la vidéo, et cliquables seulement si cela est possible (par exemple on ne peut pas mettre en pause si le flux n'est pas diffusé, ...). De plus les informations suivantes sont placées en-dessous des boutons : le type et le label de la piste, et en cas de vidéo les dimensions (largeur et hauteur) et le nombre d'images par secondes. Nous nous intéresserons aux détails de l'audio plus tard. Néanmoins, cela nécessite de distinguer les sources, ce qui est possible avec en utilisant le type.

Exercice 6 : L'objectif est de généraliser l'exercice 5, en ajoutant aussi la possibilité de capter l'audio, en utilisant un élément HTML personnalisé `<audio-video>` avec les attributs suivants :

- `source`, avec valeurs possibles : `audio`, `video`, ou les deux,

- width ou height pour la vidéo.

Les sources (audio et vidéo) doivent pouvoir être stoppées et mises en pause indépendamment. Mettre en pause l'audio revient à rendre la source silencieuse (*muted*), ce qui est géré par la propriété `enabled` de `MediaStreamTrack`, voir référence :

<https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack/enabled>

Exercice 7 : Testez votre solution avec un smartphone, et observez les sources disponibles.

Référence pour l'ensemble de l'API :

https://developer.mozilla.org/en-US/docs/Web/API/Media_Capture_and_Streams_API

2) Prendre une photo à partir du flux vidéo

Il existe deux techniques pour prendre une photo :

- Extraire une image du flux vidéo (instantané vidéo).
- Prendre une image fixe avec le mode appareil photo de l'appareil.

La 2^{ème} technique permet d'acquérir des images de résolution supérieure, et est donc à utiliser lorsque cela est nécessaire.

L'API `MediaStream Image Capture` met en œuvre ces deux techniques, mais elle n'est pas encore disponible dans tous les navigateurs.

2.1) Prendre une photo sans l'API Media Stream Image Capture

Seule la 1^{ère} technique, la plus ancienne et générique sur le Web, est possible sans cette API. Elle consiste à utiliser un canvas et son contexte graphique 2D pour dessiner une image directement à partir du flux avec la méthode `drawImage()` :

```
context.drawImage(video, 0, 0, width, height);
```

Le contexte graphique peut ensuite être sauvegardé dans un objet `Image`, par exemple au format PNG, voir la référence suivante pour plus de détails sur cette approche :

https://developer.mozilla.org/en-US/docs/Web/API/Media_Capture_and_Streams_API/Taking_still_photos.

Cette approche reste(r) donc utile pour compatibilité avec les anciennes versions des navigateurs.

Exercice 8 : Mettez en œuvre cette technique, en complétant le code de l'exercice 6 pour permettre de prendre une photo avec un nouveau bouton. Le résultat de la prise photo sera visible dans le navigateur, et l'image sera téléchargeable.

2.2) Prendre une photo avec l'API MediaStream Image Capture

Un objet `ImageCapture` est construit avec un objet `MediaStreamTrack` comme source, obtenu à partir de `getUserMedia()` :

```
const imageCapture = new ImageCapture(mediaStreamTrack);
```

L'API met en œuvre la 2^{ème} technique avec `takePhoto()`, qui retourne un `Blob`, résultat d'une seule exposition photographique, téléchargeable, stocké par le navigateur, ou source d'un élément HTML `<img>`, par exemple :

```
const img = document.querySelector('img');
imageCapture.takePhoto()
  .then(blob => {
    img.src = URL.createObjectURL(blob);
```

```
img.onload = () => { URL.revokeObjectURL(this.src); }  
})  
.catch(error => console.error('takePhoto() error:', error));
```

Avec une résolution généralement plus faible, l'API met en œuvre la 1^{ère} technique avec la méthode `grabFrame()`. Elle effectue un instantané du flux vidéo sous la forme d'un objet `ImageBitmap`, par exemple dessinaable dans un canvas :

```
const canvas = document.querySelector('canvas');  
imageCapture.grabFrame()  
  .then(imageBitmap => {  
    canvas.width = imageBitmap.width;  
    canvas.height = imageBitmap.height;  
    canvas.getContext('2d').drawImage(imageBitmap, 0, 0);  
  })  
  .catch(error => console.error('grabFrame() error:', error));
```

En plus des propriétés et fonctionnalités des sources obtenues via `MediaDevices` et/ou `MediaStreamTrack`, l'objet **ImageCapture** en fournit via les méthodes `getPhotoSettings()` et `getPhotoCapabilities()`, qui retournent un objet `PhotoSettings` et `PhotoCapabilities`, respectivement.

<https://w3c.github.io/mediacapture-image/#photosettings-section>

<https://w3c.github.io/mediacapture-image/#photocapabilities-section>

Référence pour l'API :

https://developer.mozilla.org/en-US/docs/Web/API/MediaStream_Image_Capture_API

Exercice 9 : Mettez en œuvre les deux techniques avec l'API `ImageCapture`, en complétant le code de l'exercice 6 pour permettre de prendre une photo avec un nouveau bouton. Le résultat de la prise photo sera visible dans le navigateur pour les deux techniques. Vous pouvez vous inspirer des l'exemple de Chrome <https://googlechrome.github.io/samples/image-capture/grab-frame-take-photo.html>. L'image rendue, pour chaque technique, sera téléchargeable. Vous testerez votre code sur un ordinateur et un smartphone.